Performance Analysis of Sparse Matrix-Vector Multiplication (SpMV)

Michael Bernasconi
ID Student: 267681
michael.bernasconi@studenti.unitn.it
https://github.com/Michael-Bernasconi/Sparse-Matrix-Vector-Multiplication
Department of Information Engineering and Computer Science
University of Trento

***Abstract*—–-**
***Index Terms*—component, formatting, style, styling, insert**

I. Introduction

Problem statement, why SpMV matters, and the question your investigation addresses.

- **Problem Statement:** What the Sparse Matrix-Vector Multiplication (SpMV) operation is and why it is a fundamental building block in HPC.
- **The Bottleneck:** Why SpMV performance is typically memory-bound (due to extremely low arithmetic intensity).
- **Scope and Objective:** The research question your investigation addresses. The goal is to move beyond a COO vs. CSR vs CSR-vector vs cuSPARSE evaluation and produce a comprehensive experimental study on how sparse storage formats and implementation strategies affect SpMV performance on the GPU Ampere architecture.

II. Methodology

Formats tested, CPU/GPU implementations, validation method, measurement methodology, and hardware/software environment.

- **Hardware & Software Environment:**
  - Specify the target architecture (e.g., NVIDIA A30 GPU, Ampere), CUDA toolkit version, compiler flags (e.g., `-O3`, `-arch=sm_80`), and OS.
  - Mention the use of Valgrind –¿ (only cpu cache, beacuse GPU the tools are blocked until the second deliverable)
- **CPU Baseline & GPU Kernels:**
  - *CPU Baseline:* A straightforward sequential or OpenMP implementation, explicitly stating it is used for validation and is not aggressively optimized[- 12, 38].
  - *GPU Formats:* (PSEUDOCODICE) Briefly describe the tested formats (e.g., COO, CSR, CSR-Vector, cuSPARSE).
  - *Kernels Tested:* Explain your implemented kernels: Base COO, Base CSR, and your **Optimized CSR**. Detail how you use shared memory to reduce the cost of accessing global memory and how warp-level primitives improve load balancing.
- **Measurement Methodology:**
  - *Consistency:* To ensure numerical consistency between write-only formats (CSR) and atomic-accumulate formats (COO), state that an explicit output vector reset was included within the benchmark loop.
  - *Variability Mitigation:* Explain that runtime is isolated strictly to kernel execution, ignoring one-time setup costs like file parsing. To mitigate shared environment noise, each benchmark performs 5 independent runs, reporting the best-of-5 (minimum execution time) as a justified stable statistic.
- **Performance Metrics (Formulas):**
  - State the formula for computational throughput (GFLOP/s), defining the standard estimate of 2 floating-point operations per non-zero (NNZ) element.
  - Define the formula for Effective Bandwidth (GB/s).
  - State the Theoretical Peak Bandwidth of the target GPU (e.g., $\sim$933.1 GB/s for the A30) as the theoretical upper bound.
  - Provide formulas for Cache Hit/Miss rates as extracted from the profiler.
- **Correctness & Validation:**
  - Describe the validation function `validate_results(float *y_cpu, float *y_gpu, int M)`.
  - Since `Float32` is used, explain the tolerance-based comparison method (e.g., $\epsilon = 1e - 3$). Implement a `for` loop over the entire vector: if $|y\_cpu[i] - y\_gpu[i]| > \epsilon$, print “FAIL”, otherwise “PASS”.
  - *Theoretical Explanation:* Explain why validation depends on the representation method. In parallel architectures (GPUs), the thread execution order for additions (e.g., via `atomicAdd`) is non-deterministic. Because floating-point arithmetic is non-associative, different summation orders inevitably generate slightly different rounding errors.

III. Dataset

Short characterization of the 10 SuiteSparse matrices and the random dense vector input.

- **SuiteSparse Selection:** Justify your dataset selection. State that the 10 matrices were carefully chosen to cover different regimes in size, sparsity structure, and row-length variability, including both highly structured and clearly unstructured matrices to stress different access patterns.
- **Summary Table:** Insert a structural summary table. Recommended format: `Matrix Name | M (Rows) | N (Cols) | NNZ | Average NNZ per row | Row length variance | Description`.
- **Input Vector:** State that for every experiment, a randomly generated dense `Float32` vector of compatible size is used, keeping the random seed fixed (e.g., `random(42)`) to ensure reproducibility.

IV. Results

Plots/tables for runtime and FLOPS/GFLOP/s; optional memory/cache indicators. *(Note: Visual data presentation only; formulas are established in Methodology).*

- **Throughput & Runtime:** Include bar charts comparing GFLOP/s across the 10 matrices for Base CSR, Base COO, Opt CSR, and cuSPARSE. Include kernel-time plots displaying the measured temporal variability (e.g., standard deviation or min-max bounds).
- **Bandwidth Plots:** Create Effective Bandwidth (GB/s) charts. Add a horizontal line representing the Theoretical Peak Bandwidth (e.g., $\sim$933.1 GB/s) to visually demonstrate how close the implementations are to being memory-bound.
- **Memory/Cache Indicators:** Provide charts or tables (using data from `valgrind`) showing cache hit/miss behavior and global memory accesses.
- Nonostante le ottimizzazioni del kernel, il TTS rimane dominato dal caricamento I/O per le matrici meno dense

V. Discussion

Interpret the results in terms of format properties, matrix structure, and memory behavior. Connect the measured behavior to algorithmic techniques and matrix characteristics.

- **Load Balancing & Matrix Structure:** Do not simply state “kernel X is faster”. Explain *why*. Discuss how row-length regularity and variance affect performance. For instance, precisely explain under which structural conditions (e.g., highly irregular matrices) CSR-Vector outperforms CSR-Base by mitigating load imbalance.
- **Memory Behavior & SM Utilization:** Address how close the results are to the memory bound and what evidence supports this. Use the `valgrind` profile cache for cpu.
- **The cuSPARSE Advantage:** Analyze the performance gap with cuSPARSE, theorizing its use of adaptive load-balancing techniques, dynamic partitioning, or undocumented heuristics tailored to specific matrix regimes.

VI. Conclusion

Main lessons learned, limitations, and practical takeaways.

- **Main Lessons Learned:** Summarize which observations are intrinsic to the mathematical structure of the matrix itself, and which are artifacts of specific implementation choices.
- **Practical Takeaways:** Clearly state the optimal use-cases. Explain exactly under which matrix characteristics (e.g., low NNZ/row, high variance) and memory-access conditions a specific kernel or format ceases to be advantageous and another should be preferred.
- **Limitations:** Briefly mention study limitations (e.g., exclusively testing `Float32` arithmetic or specific Ampere architecture constraints).

References

References

[1] Gao, J., Liu, B., Ji, W. and Huang, H., 2024. A systematic literature survey of sparse matrix-vector multiplication. arXiv preprint arXiv:2404.06047.

[2] Chu, G., He, Y., Dong, L., Ding, Z., Chen, D., Bai, H., Wang, X., and Hu, C. Efficient Algorithm Design of Optimizing SpMV on GPU. In HPDC ’23, 2023.

[3] Bell, N., and Garland, M. Implementing Sparse Matrix-Vector Multiplication on Throughput-Oriented Processors. In SC ’09, 2009.

[4] Greathouse, J. L., and Daga, M. Efficient Sparse Matrix-Vector Multiplication on GPUs Using the CSR Storage Format. In SC14, 2014.

[5] Liu, W., and Vinter, B. CSR5: An Efficient Storage Format for Cross-Platform Sparse Matrix-Vector Multiplication. In ICS ’15, 2015.

[6] Merrill, D., and Garland, M. Merge-Based Parallel Sparse Matrix-Vector Multiplication. In SC16, 2016.

[7] M. Young, The Technical Writer’s Handbook. Mill Valley, CA: University Science, 1989.